

delay 1-sec

mov R2, #08 → ceiling of 7.68

R1, #255

R0, #255

reload R1: → mov
reload R0: → mov

↑
innermost

D-JNZ R0, here → $[(2 \times 255) \times (255)] \times 8$

D-JNZ R1, reload-R0

D-JNZ R2, reload-R1

but
end

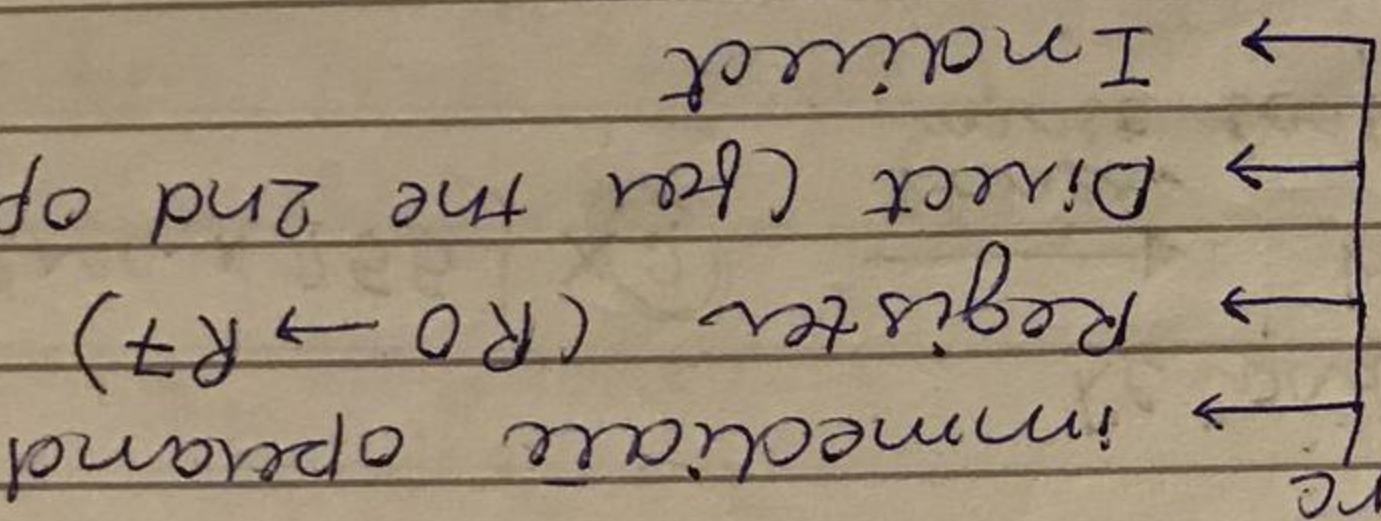
25-09-2017

Mid-2:

Arithmetic Instructions:

Addition: add

dest, src



mov A, #255 } Immediate source
add A, #3DH }

add A, 3DH } direct

↓
A k address pay jao, aur waham jo num hai usay A k 5th add karana hai.

(27)

Input using P0, P1 and display output on P3
Outside World Data:

mov A, P0
mov B, P1
add A, B
mov P3, A

← Reading the port
(can be any register R0, or any location)

mov A, #2FH
add ~~mov~~ A, #3DH
A ← 6CH

considering them signed numbers,

MSB=0

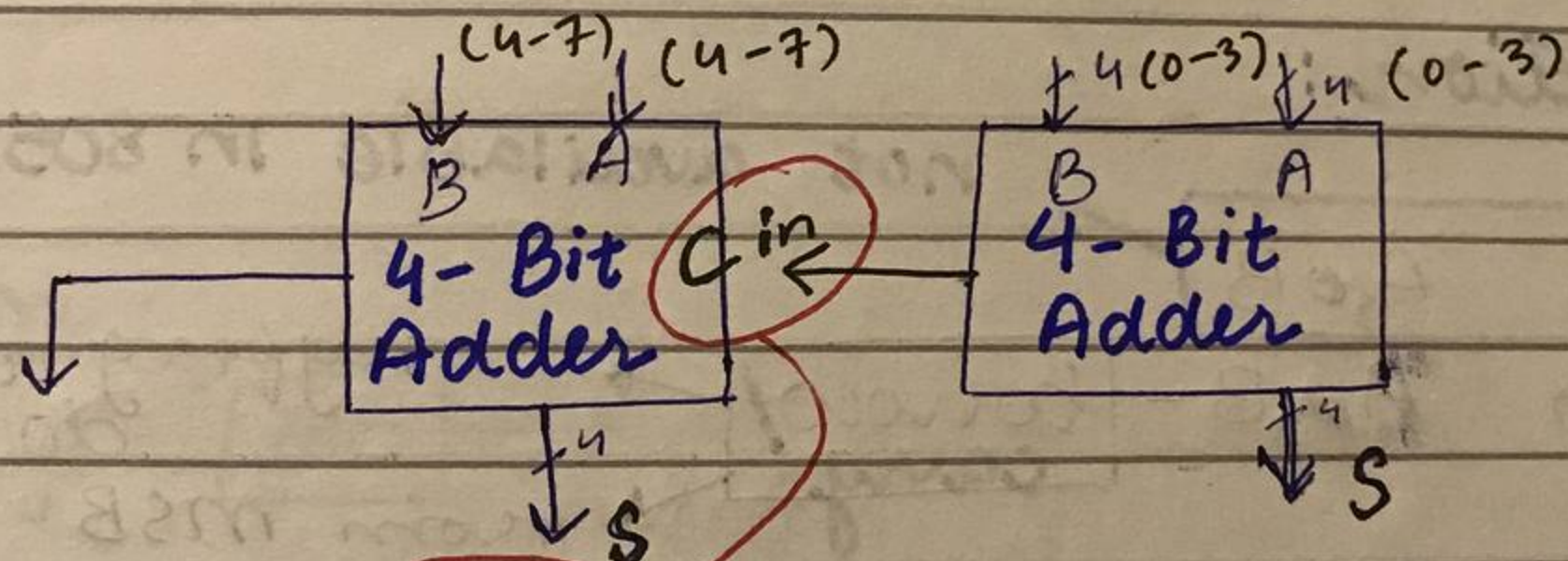
⇒ +ve number CY=0
AX=1
OV=0

Auxiliary carry

0010	1111
+ 0011	1101
0110	1100

upper nibble Lower nibble

If we want to add 8 bit data:

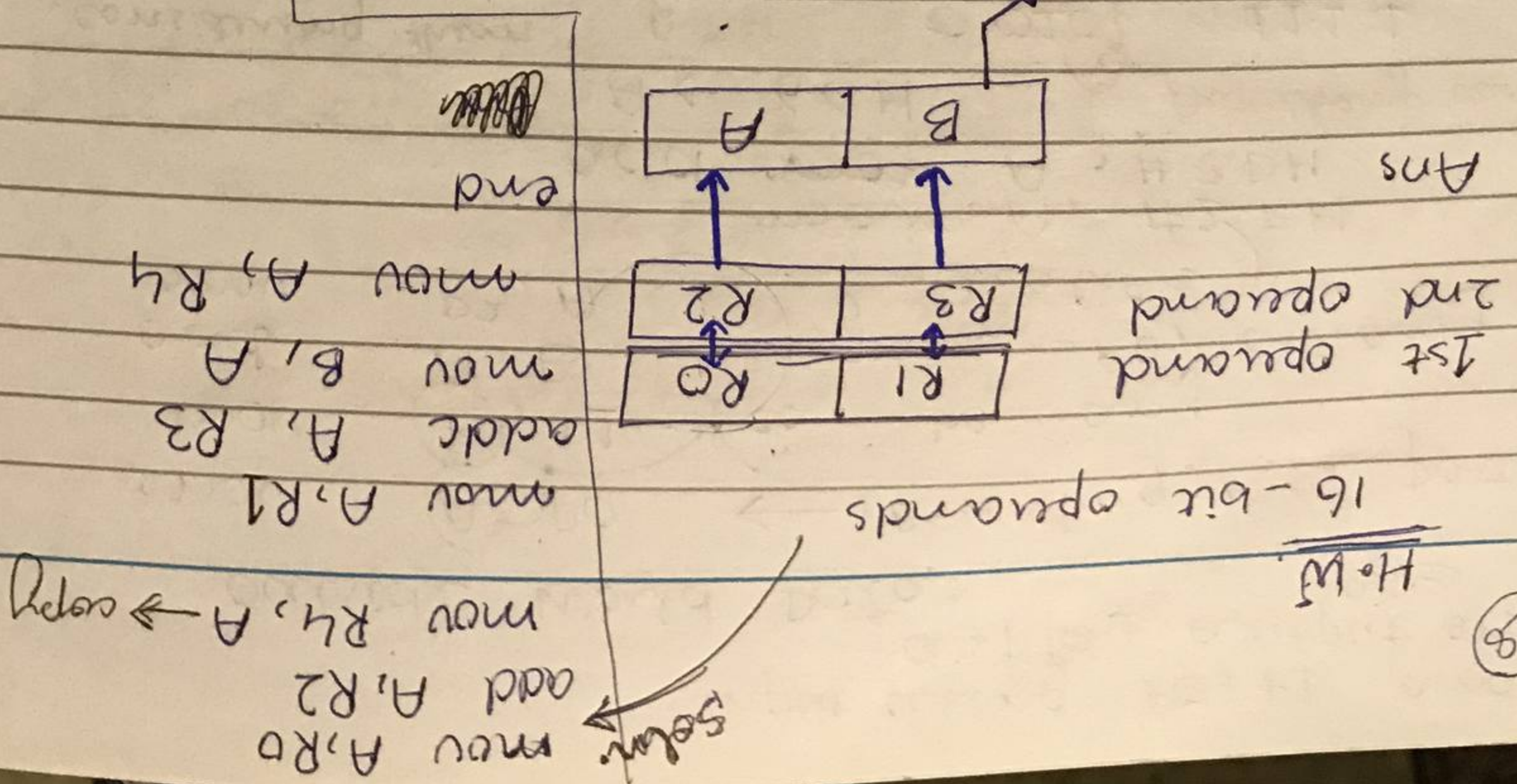


addc A, #63H
add carry

Result will also be displayed in 2 Registers.

H.W.

16-bit operands



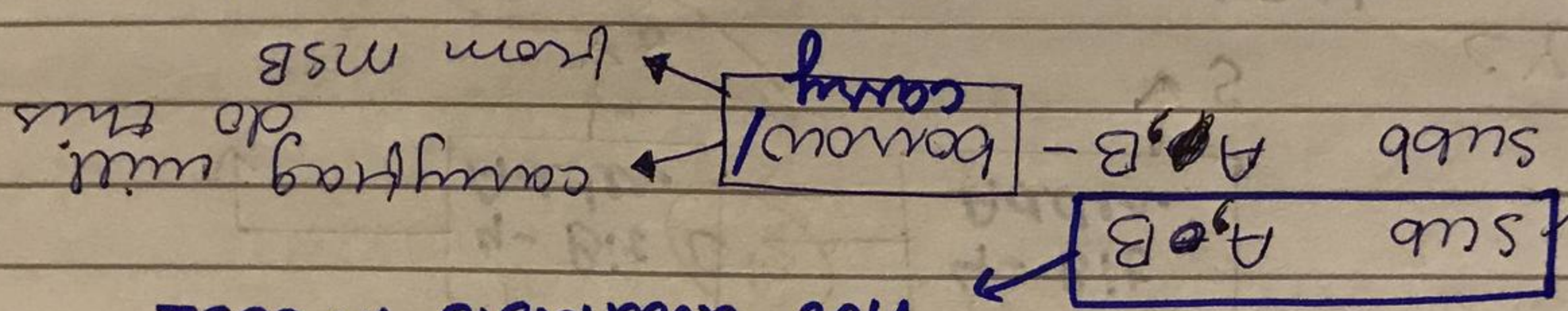
if carry appears here
CY flag will handle it.

R50, R51, C

Auxiliary carry → write location

Subtraction:

not available in 8051



if you want to execute this statement then first clear carry. i.e. $A - B - 0$

ACALL:

orig 0x0
acall loop1
setb p1.0

III 1000 0000 1000

org 0x101F
loop 1:

★ Lower 11 bits of
loop are stored
shuru ki 5 bits will
be copied from
Program Counter.

27-09-2017

Wednesday

11

Count the no. of ones from Port^P1 and display the result on Port 2.

min value → max value

Soln. on next pgs

2nd conditional Jump:

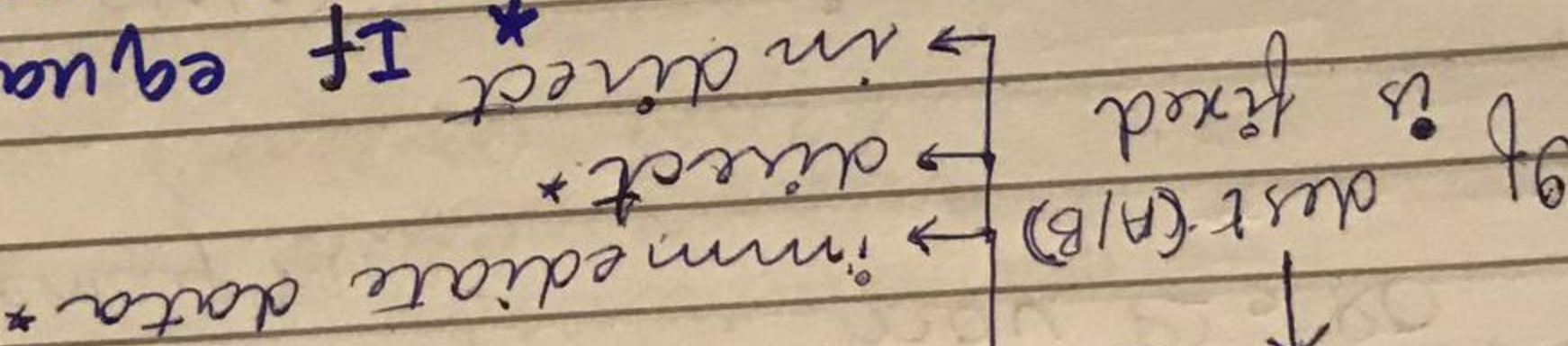
CJNE

compare jump if not equal

- shift left
 - shift right
 - rotate left
 - rotate right
- can be done using carry flag

Logical operators

① CJNE A, #data, label



* If equal, execute the immediate next instruction, else go to LABEL.

② CJNE Rm, #data

③ CJNE Ri, #data

Ri jahan point kar raha hai, wahan go operand hai uska jao.

2 2 | 2
2 1 | 1
0 0 | 0
1 1 | 1

↑ Quotient

0 0000 A 1010 B

Soln 1:

not-equal:

inc R0

jmp checknext

mov A, P1

mov R0, #0; counter

mov R1, #8

divide again:

(of 0's)

mov B, #2

DIV AB

if B==0
executing
next time

B may overwrite

hojama hai

isliye remainder

check kar rhy

pehlay hi.

CJNE B, #0, not-equal

check next:

DJNZ R1, divide again

mov P2, R0

Soln 2: (incomplete)

→ Explanation:

How
using
bitwise
xor

cjne A, direct, label

A, #data

copy of A data

- ## data

* ~~gm~~

Result

if (dest >= src)

→

C = 0

else

C = 1 → dest < src

① **jc label** → jmp to the label if $c = 1$

② **jnc label** → jmp to the label if $c \neq 1$

③ **jb P1.0, label** → jmp if bit is '1'
P1.0, label
e.g.

jnb 00H, label → bit addressable area of RAM (2nd portion)

④ **jnb P1.0, label** → jmp if the bit is not set/
P1.0, label

if bit status is ZERO

⑤ **jbc P1.0, label** → again bit ki value 1 hai to label pe jaao and clear the bit, too
P1.0, label
modifies

JZ label → jmp if $A = 0$
JNZ label → jmp if $A \neq 0$

Tests for Accumulator
For the test of turn of any.

Q- Check the pattern 0000 1010B using bit Testing.

(34)

Oct 2, 2017

Soln2

```
org 0x0
mov R0, #0
jnb P1.0, checknext
; if it is set, go to label
```

checknext:

inc R0

jnb P1.1, checknext-1

⋮

} it will inc
R0 everytime
whether bit is 1 or
0

```
main: org 0x0
mov R0, #0
jnb P1.0, checknext
```

inc R0

checknext:

jnb P1.1, checknext-1:

inc R0
checknext-1:

jnb P1.2, checknext-2:

⋮

For every bit

inc R0

checknext-8

jnb P1.7, display

inc R0

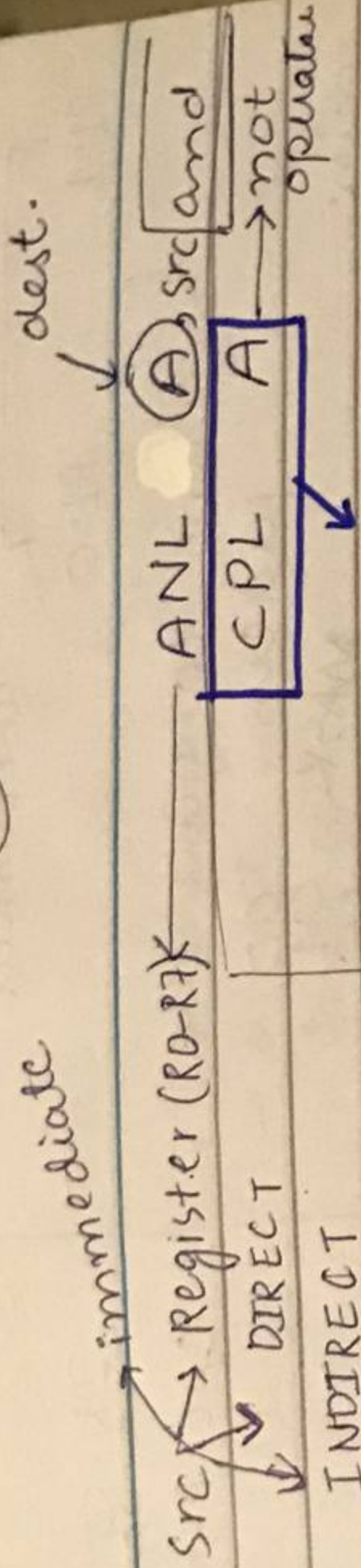
display:

mov P2, R0

jmp main (ta k
loop
chalta rhe)
end

→ if '0' go to display

35



only acc is allowed

Bit-wise [AND]

mov A, #0FH
and A, #2BH

0 F
[0000] [1111] B → A
[0010] [1011] B → imm.
2 B
0000 1011 → A
→ Result

mov A, #1AH
cpl A
A ← 1110 0101 B

Q- P1 pay 2 bit data

dena hai (Random) at P3.3
2 kHz = 250 μs
4 kHz = 125 μs
6 kHz = 83.3 μs
8 kHz = 62.5 μs

Single sq. wave.

0	0
0	1
1	0
1	1

f_c = 12 MHz • Delays

Soln.

For 2 kHz sq. wave

$$\frac{1}{2k} = T = 0.5 \text{ ms}$$

= 500 μs → Total time Period

For the generation of 50% D.C. → Duty cycle
sq. wave of 50% D.C. → no DC was given, assume it

$$\frac{500 \mu s}{2} = 250 \mu s \rightarrow \text{delay}$$

binary switches,
debounced

P1.1 P1.0

→ want to give the

pattern 00

mask 01

10

11 masking

→ transferring

(the status

of bits only)

2

main:

mov A, P1

and A, #00000011B

cjne A, #0 check-1

jmp 2k sq. wave

check-1:

cjne A, #1, check-2

jmp 4k-squawave

check-2:

cjne A, #2, check-3

jmp 6k-squawave 0V

check-4:

cjne A, #3, ~~check-5~~

jmp 8k-squawave

H.W. 2

Repeat the above
problem using jb and
jnb. (P1.0, P1.1 ko
aag test karin)

26

2K sq. wave: Routine

cpl P3.3 → for conti.

square wave

call delay 250us

jmp 2K sq. wave

no option to

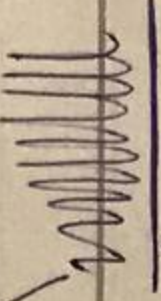
come out of this loop

250us * 12 = 3s

jab portb
byte treat
karin gay
to maski-
NG karin
paragi



5V



→ To stabilize it, we use

Debounce & the process is called

debouncing.

For copy
→

ORL → OR logical

ORL A, SRC → immediate
 → direct
 → indirect
 → Register (R0-R7)

e.g.

MOV A, #2FH
 MOV 30H, #0F0H
 ORL A, 30H — (Direct)

0010 1111 : A
 1111 0000 :

 1111 1111 → A

ORL can mask can any pin by forcefully setting the pin.

MOV A, #23H

23 ko karana 0010 sath
 XOR hai k

XOR

A, # 0010 0000B

7 6 5 4 3 2 1 0
 0 0 1 0 0 0 0 0
 want to toggle it

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

0010 0011 : A
 0010 0000 : (+)

0010 0011 → A

XOR can act like a buffer if one of its input is set at gnd.

4-oct-2017

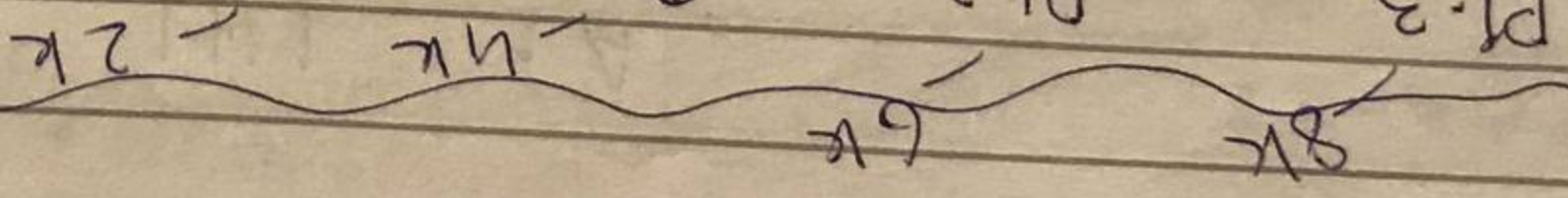
```

org 0x0
jb P1.0, jump3
jb P1.1, jump3
call 2K-HZ
    
```

```

jump:
jb P1.1, check-3
call 4-KHz
    
```

11 ko
dual kuga



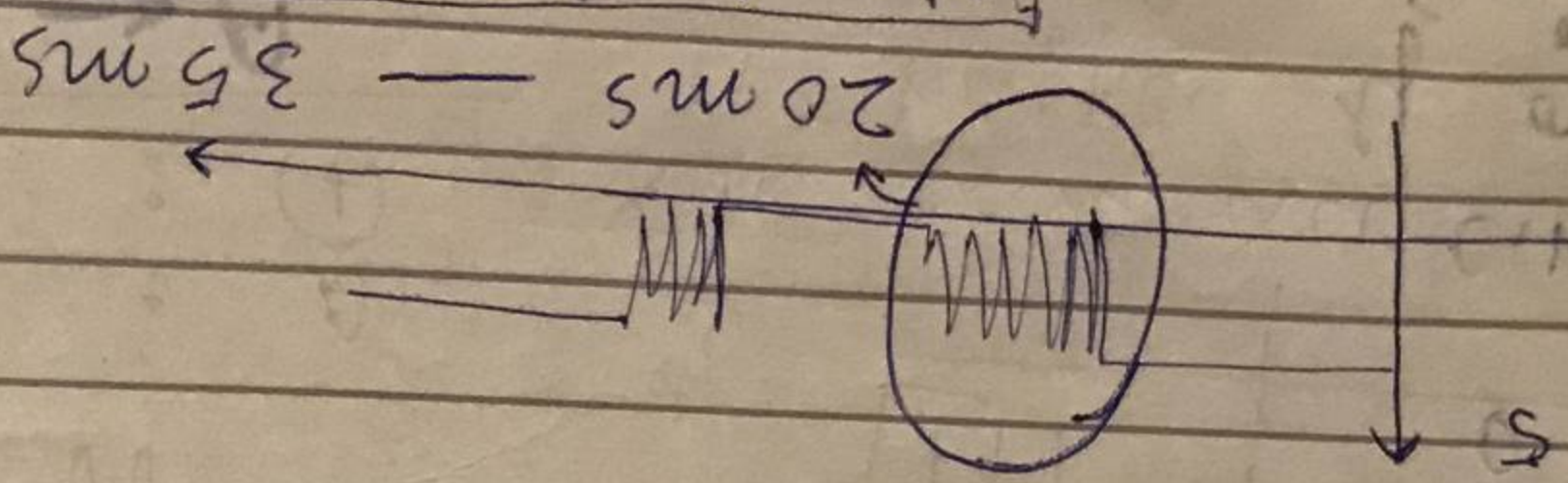
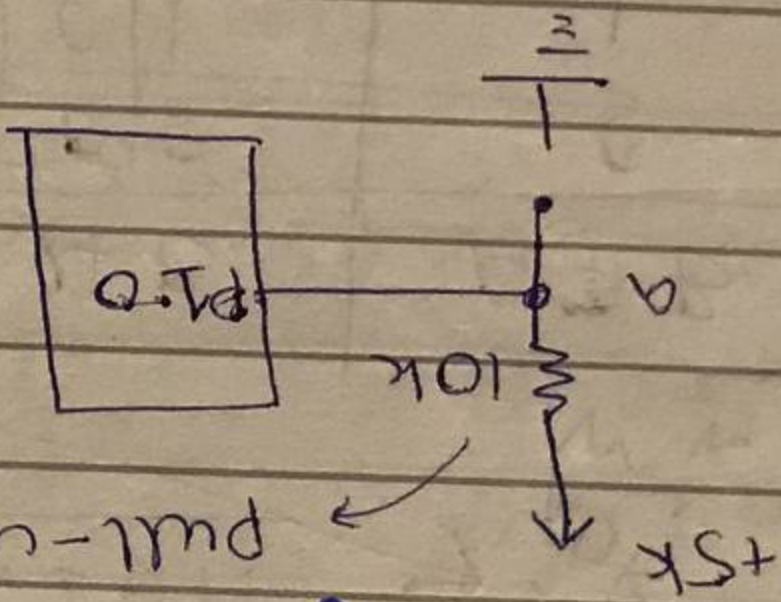
```

org 0x0
mov A, P1
and A, #0FH
cjne A, #0001011B, checknext2
jmp sq-wave2k
    
```

cjne is the best way of doing it.

pull-up Resistor because when switch is

opened a and +5V are at same level, because the drop across 10k is 0V, so it feels like we have pulled 'a' to its max. value (5V)



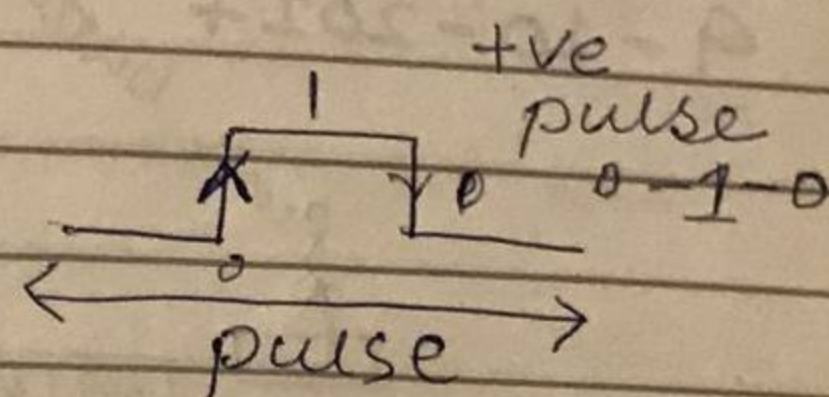
Normal value = 25ms of debouncing

To stabilize the bouncing effect

2 ways:

- (1) Software debouncing (delay)
- (2) Hardware debouncing

Software debouncing:



Q Suppose if we get a pulse at P1.0 then do the following task.

$$P2 = (R1 + R2)$$

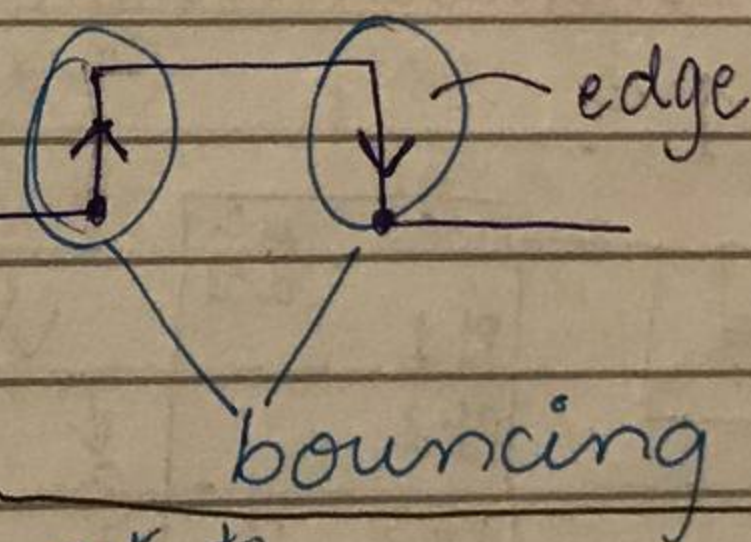
here: jnb P1.0, here
OR

jnb P1.0, \$
* call delay 25ms stay here (bouncing ki wjase)

jnb P1.0, \$ (jab tak 1 hai yaheen rho)

↓
jese hi Zero hoga, it will come out to add the content of R0 & R1

First observe the pulse (0-1-0)
→ pulse triggered



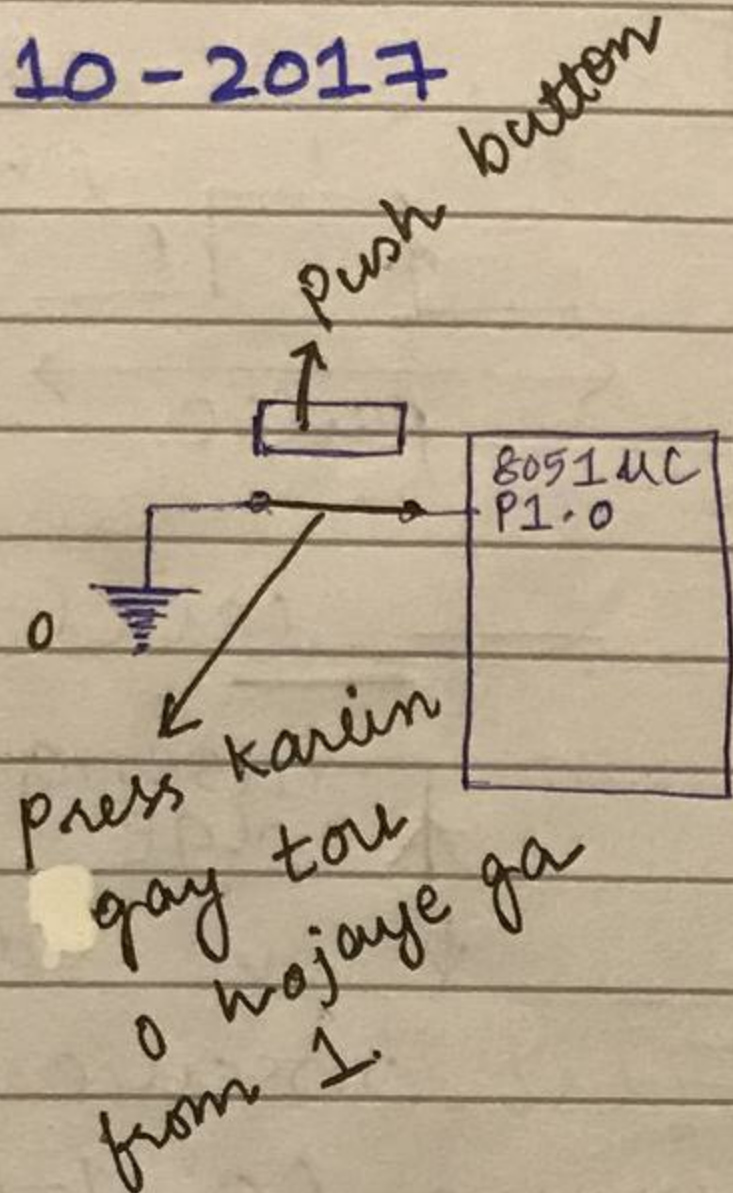
call delay - 25ms // optional here kyunke hum nay read nhi karna iska status baad mein.

```

mov A, R1
add A, R2
mov P2, R2
    
```

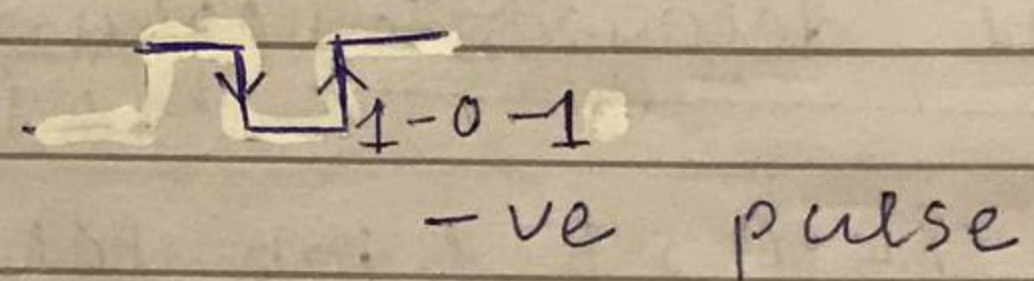
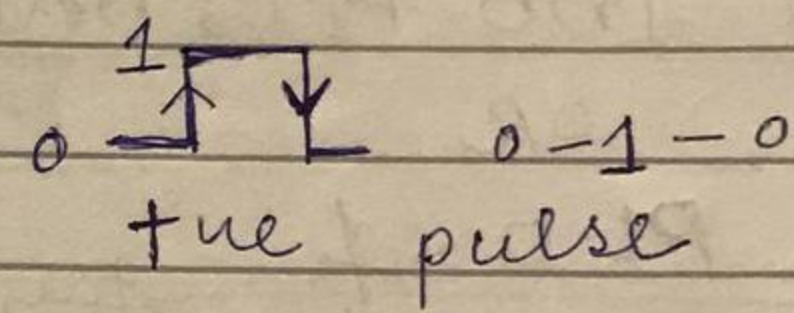
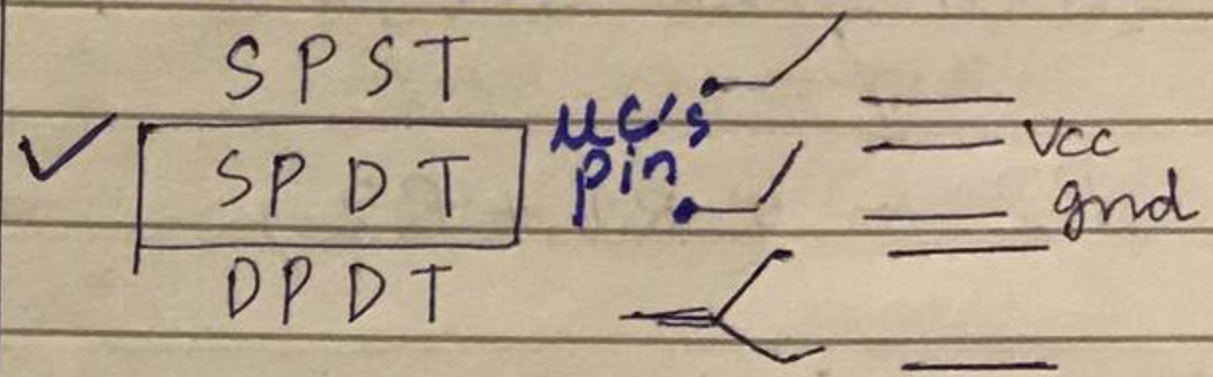
9-10-2017

Hardware Debouncing

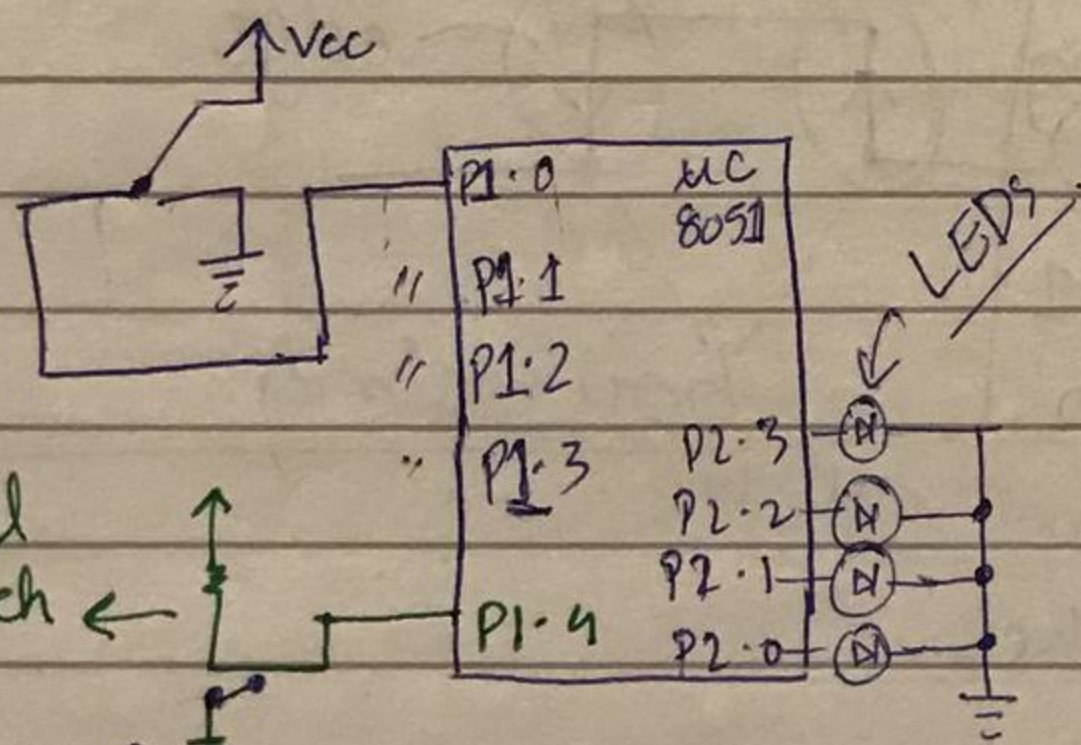


Basic concept
Momentary switches:
→ Pushed button

Latched Switches



4-bit downcounter:



Display the value you entered, delay of 1s, dec, display jb tk zero nhi ata. zero ko b display karna hai.

(41)

$$8+2+1=11$$

Soln. 10000 $\overset{3}{1}\overset{2}{0}\overset{1}{1}\overset{0}{1} \rightarrow P1_{in}$

main:

mov A, P1

★ call delay - 25 msec

anl A, #0FH

 // 1011

└─ masking

Taa k dubara
port read
karni ho
tou masla
na hou.

not-zero-yet:

mov

P2, A

call

delay 1-5:

dec A

cjne A, #0, not-zero-yet

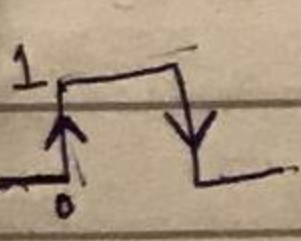
mov P2, A (to display zero)

jmp main (dubara input leay rhe)

Soln. 2

With control switch

main:



{ jnb P1.4, \$ (zero level chal
 call delay 25-msec raha hai) / polling
 jnb P1.4, \$ (open switch
 hai abhi)
 }

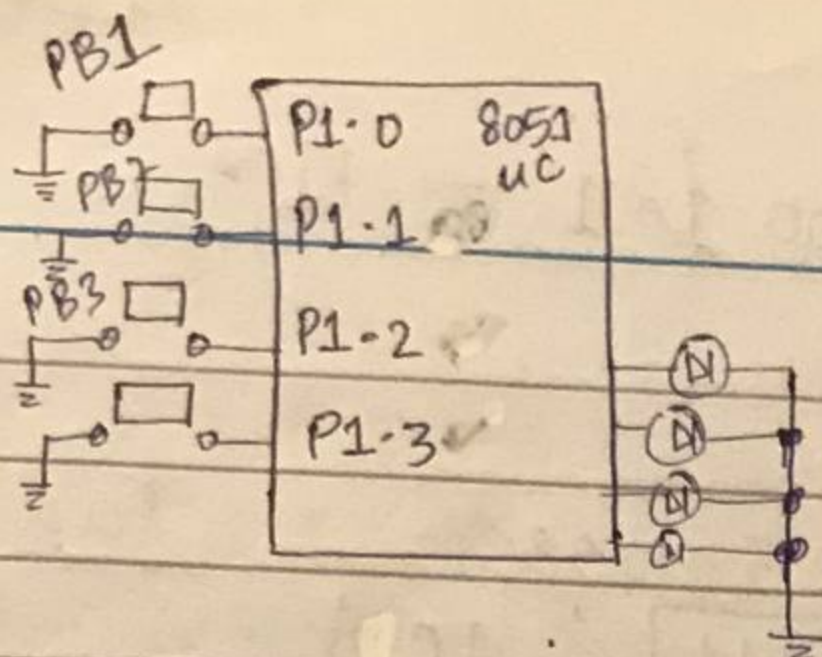
mov A, P1

mov A, #0FH

PC will
stay here

pin ko beth k
uska status
dekh rha hai.

upar wala code hai same.



Momentary switches

We don't use push buttons to

give data, they're used to trigger the data.

(a)

Daba k rakhna hai?
Daba k chorna hai?

Q - PB1, PB2, PB3
(Mechanical switches)

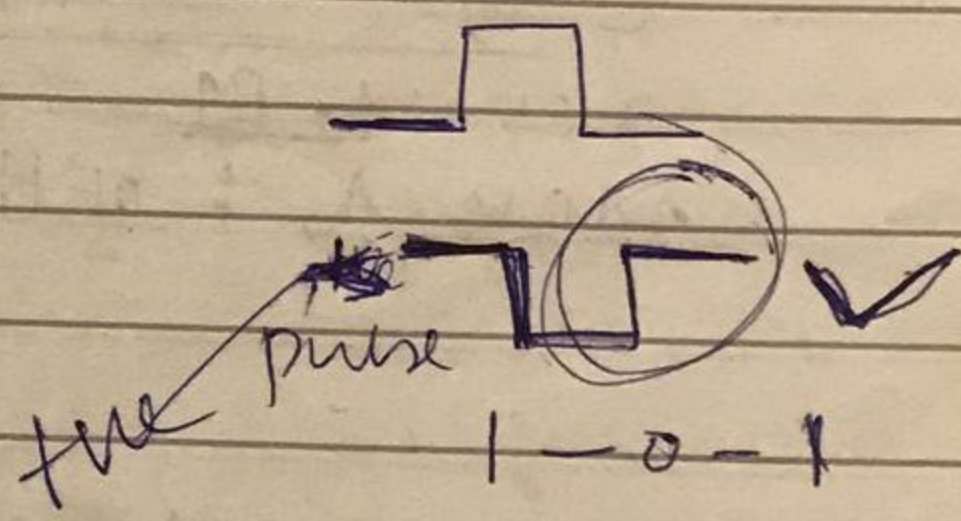
Pressed the pulse
PB1 $\rightarrow$ P2 $\leftarrow (R0 + R1)$
PB2 $\rightarrow$ P2 $\leftarrow (R0 / R1)$
PB3 $\rightarrow$ P2 $\leftarrow R0 \rightarrow R1$

(b) complete
press it
and then
release it.

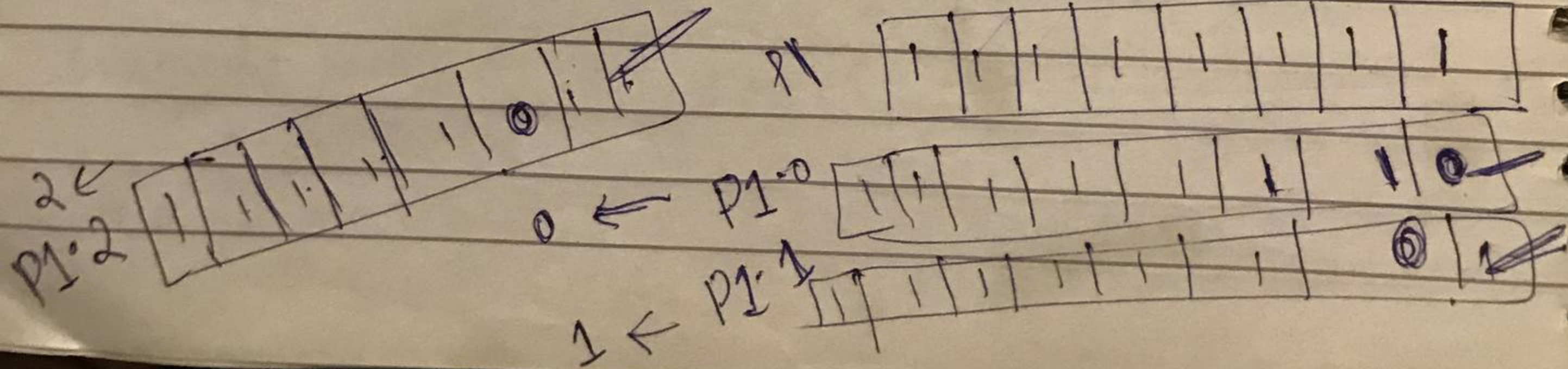
Swap kardon
(temp) //
Acc lay low

Soln

~~jnb P1.0, \$
call delay 25-msec
jnb P1.0, \$
jmp add~~



CTNE P1.



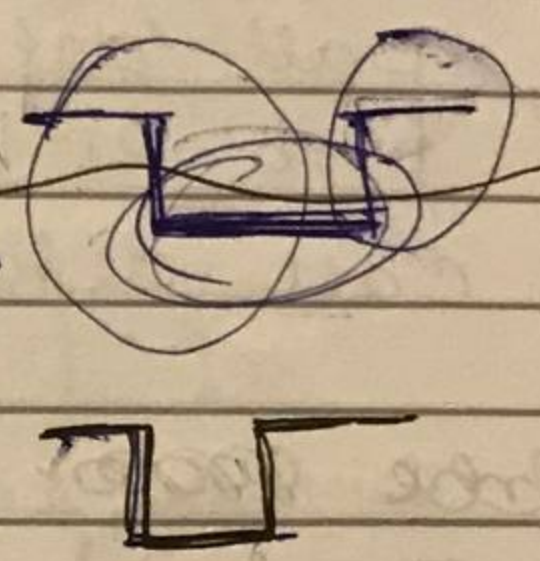
~~jb P1.0, \$~~
~~call delay 25ms~~ ~~call delay 25ms~~

2 1 0
1 0 1

mov A, P1
and A, #0111

cjne P1, #6, label1 (P1.0) PB1
cjne P1, #5, label2 (P1.1) PB2
cjne P1, #7, label3 (P1.2) PB3
jnb, P1

mov P2, #0
here: mov A, P1
and A, #0000 0111
jb P1.4, \$
call delay 25ms
jnb P1.4, \$



cjne A, #6, add; PB1 (P1.0)
cjne A, #5, div; PB2 (P1.1)
cjne A, #7, swap; PB3 (P1.2)
jmp here

add:

mov A, R0
add A, R1
mov P2, A
jmp here

div:

mov A, R0
mov B, R1
div AB
mov P2, A
jmp here

swap:
mov R0, R1
mov R1, R0
mov R0, 01H
mov R1, 00H
mov P2, A
jmp here
end

```

org 0x0
mov R0, #23
mov R1, #10
mov P2, #0
jb P1.0 $
call delay-25sec

```

Task 1:

```

call delay-25sec

```

jb P1.0, check-PB2

check-PB2:

```

mov A, R0
add A, R1
mov P2, A

```

jmp check-PB1

jb P1.1, check PB3

```

call delay-25msec

```

```

jb P1.1, $

```

```

call delay-25msec

```

Task 2:

```

mov A, R0
mov B, R1

```

```

div AB

```

```

mov P2, A

```

```

jmp check-PB1

```

check-PB3:

jb P1.2, check-PB1

```

call delay 25 msec

```

```

jb P1.2, $

```

```

call delay-25 msec

```

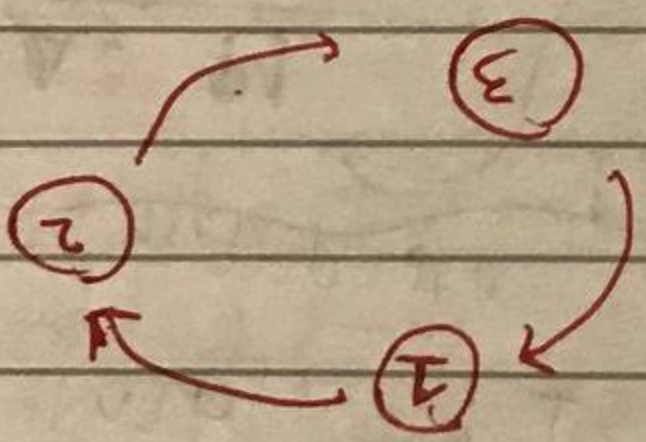
Task 3:

```

mov A, R0
mov B, R1
mov R0, B
mov R1, A

```

Perform a task,
then go to start
and check.

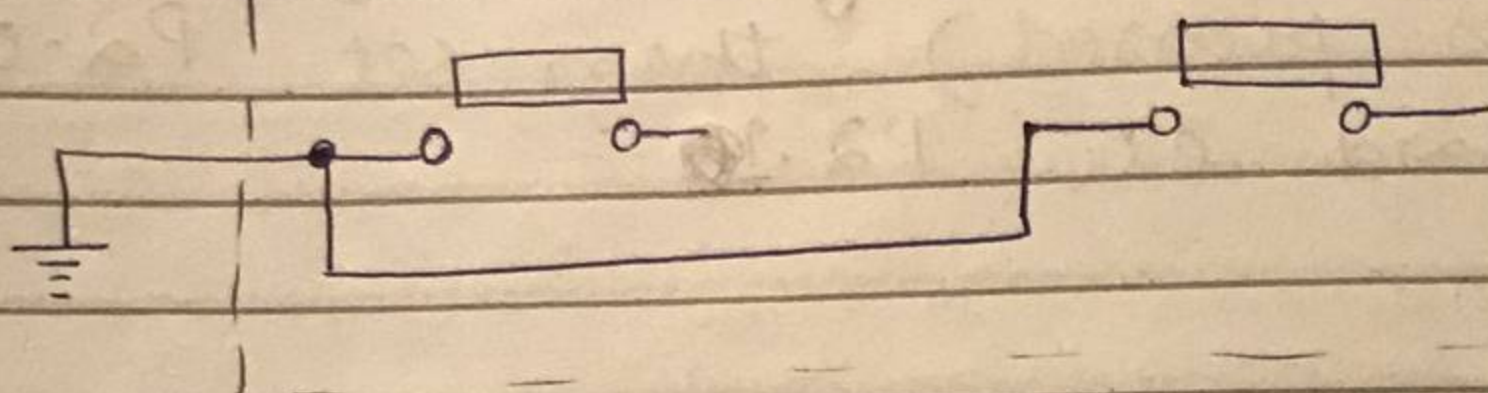


45

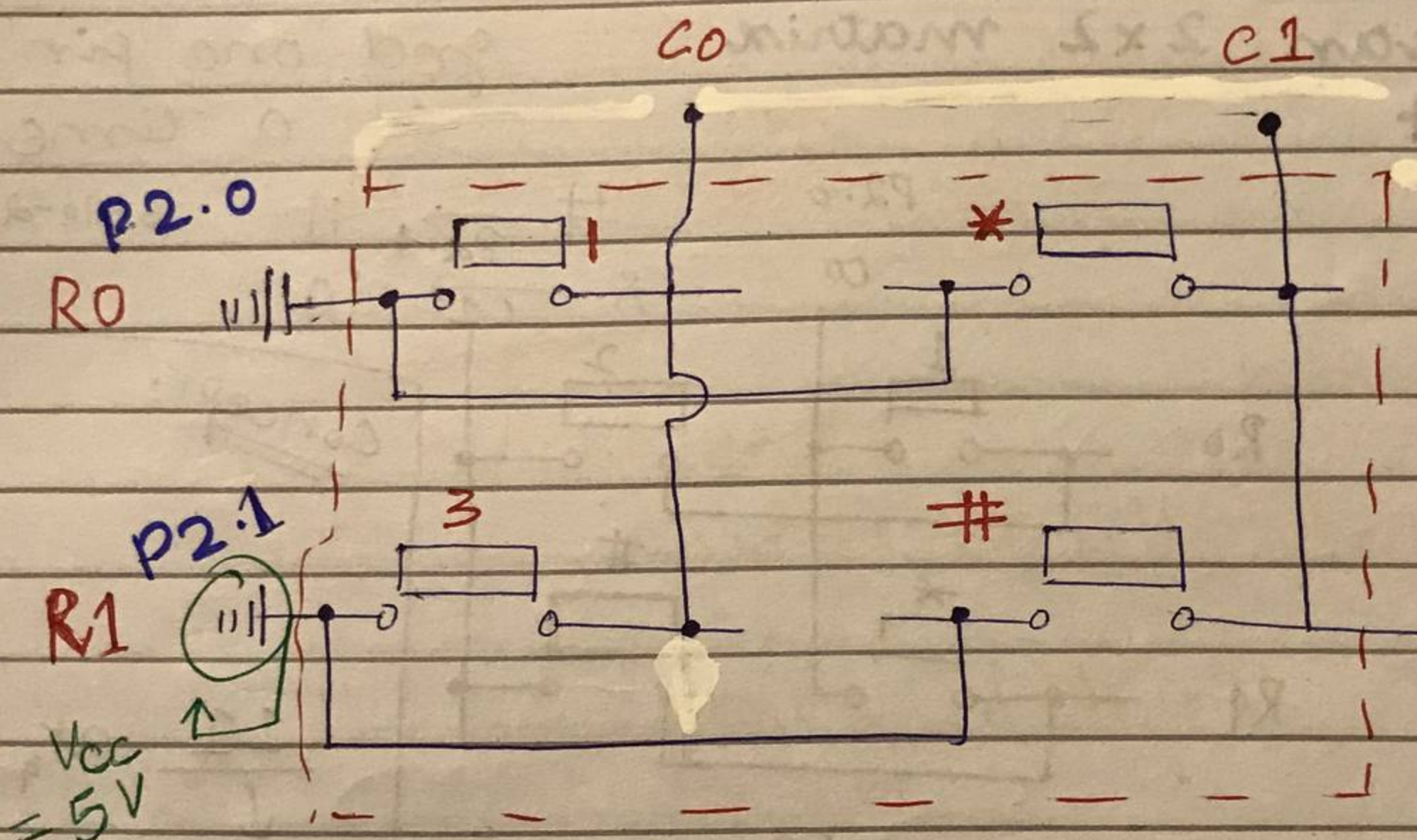
```

mov P2, A
jmp check-PB1
end

```



1x2



2x2 matrix

It can't differentiate between pressed and unpressed key

If R1 is ground, it can't diff. b/w ① and ③ now, how diff nhi ho saktin.

pg 1

(46)

I karnay "and" nhi karna. (Baari baari differentia hai, ta k row differentia hai, ta k row)

To row and hogi uska output aayega columns pay.

set $P_{2.1}$ and $P_{2.0}$

agar c_0 and c_1 pay 11 aye

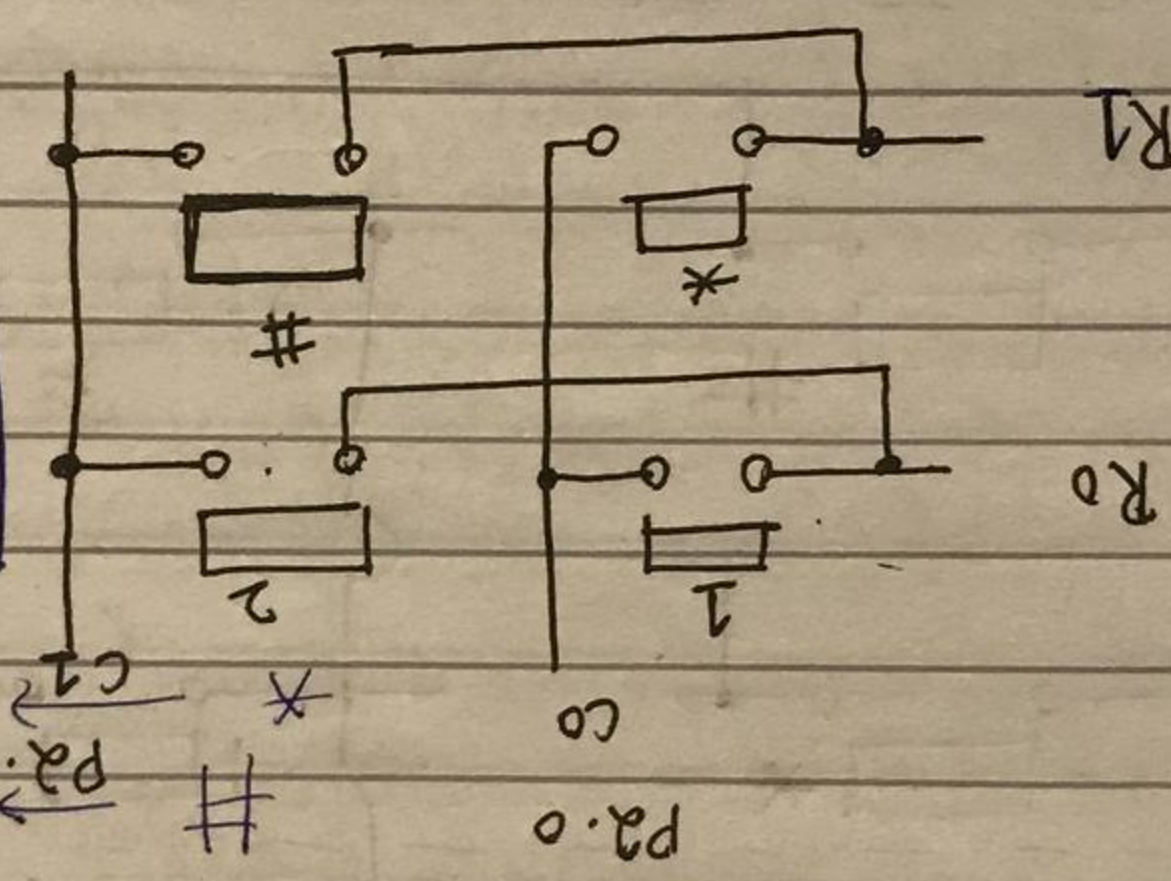
(nothing is pressed) then set $P_{2.0}$ and c_1

HW: Scan 2×2 matrix

11-10-2017

and one pin at a time.

Soln.



$P_1 \rightarrow$ rows
 $P_2 \rightarrow$ col

C_1 $P_{1.0}$

16 g jnb say hojaye ga

11 at $P_{2.0}$ nothing is pressed.

jis row ko aad karna hai, usay 0 kadeus bag!

Sab 1

concept:

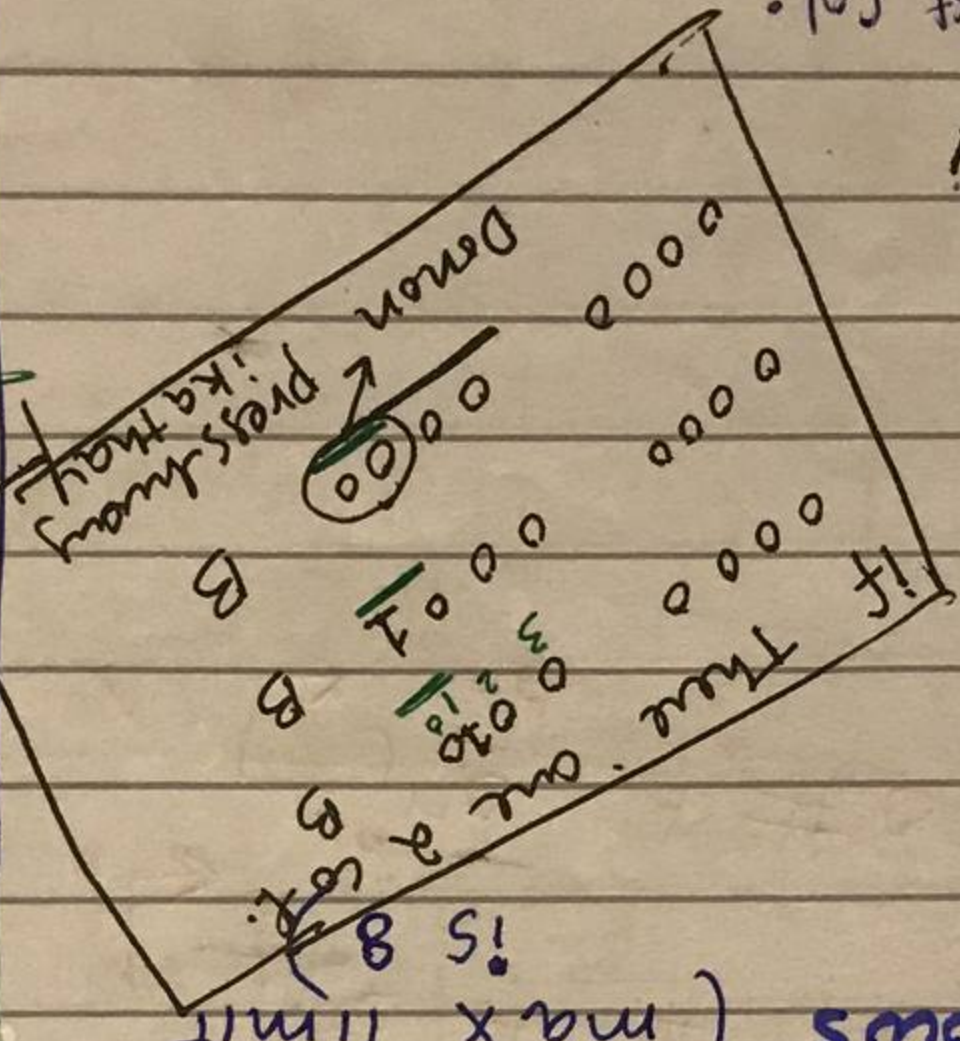
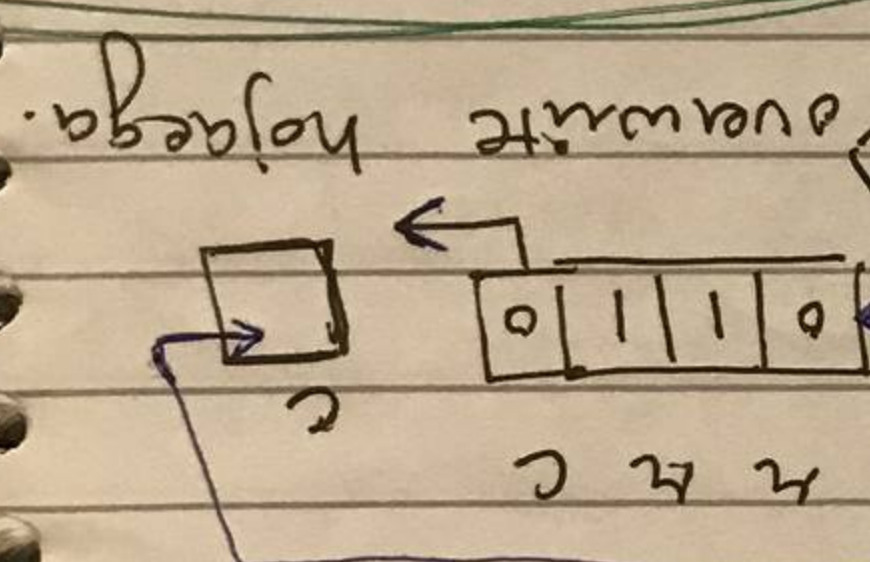
$P_{2.1}$ 11 c_1 10

16-10-2017

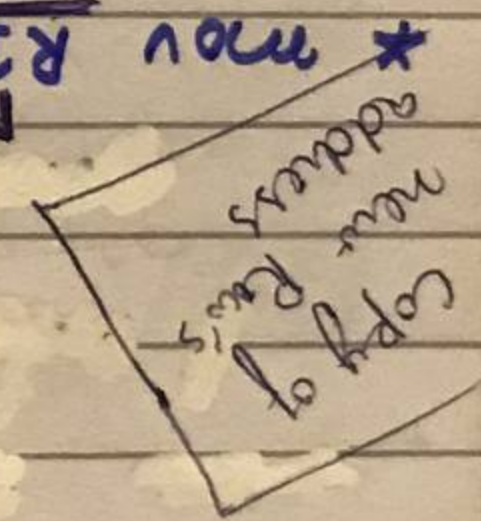
Button Release karna hai
 say pehlay karna hai find

inc find-key
 inc R3 // next col.
 cjne R3, #02, key pressed
 jmp main

*44C
 key pressed:
 delay-25ms
 A
 debouncing



inc R2
 mov R1, 0
 cjne R2, #2, again
 jmp main



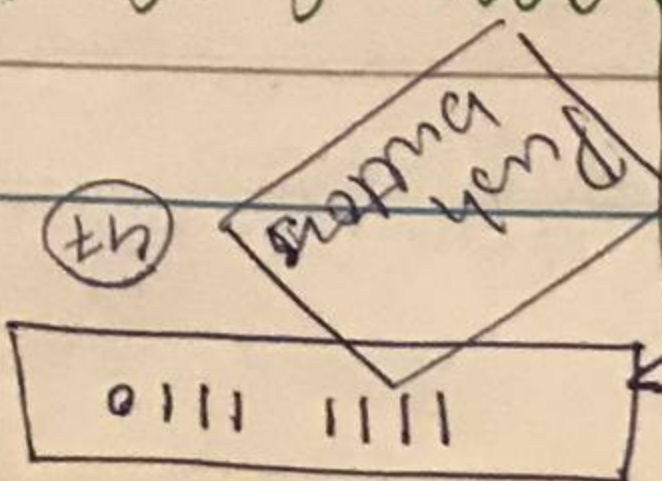
ba bar carry
 set karna
 paray ga

columns read karte
 hain is kay bad

again: mov a, R1
 mov P1, a
 mov A, P2
 (col. ko read kiah)

key pressed

A, #0000001B
 A, #0000001B
 A, #0000001B



Circuit
 jump pg
 1

Press karte hai
 main: mov R3, #0
 mov R2, #0
 org 0x0

Push buttons
 47

ko ground.

Katay hair.

find-key

and set For DPTR ROWS
 @ DPTR + columns,
 $\frac{R_3}{\downarrow}$

column Index

✓ 21#

Find Key

R2, #0 - next row

Mov A, DPTR, R3
MVC @R3, A + DPTR

R4, A
gar.

round

$r_2 = 1$
 $r_3 = 0$

after
 2×2

dy-dy's
 matrix's

is
 ved 

→ Row 0: DB
→ Row 1: DB

Evapor

if
not Row 0 is grounded

Remove error
write characters
in Quotation mark
or write their
ASCII codes.

ASC II codes

Find key: (Not generalized)

```

cjmp r2, #0 next row
mov  DPTR, #row0
mov  A, R3
movc A, @A + DPTR
mov  R4, A

```

~~Release-key~~

next-row:

```

mov  DPTR, #Row 1
mov  A, R3
movc A, @A + DPTR
mov  R4, A
jmp  release-key

```

49

Dis simulation
is simple

2

For 2x2

release-key (next row's column)

• 25-msec
could be done
by jmp r2, #7, jmp
cjmp r2, #7, jmp
cjmp r2, #7, jmp

using Accumulator

* only a

a $++$ a

$++$ a

$++$ a

$++$ a

